



ADVANCED DATABASE SYSTEMS

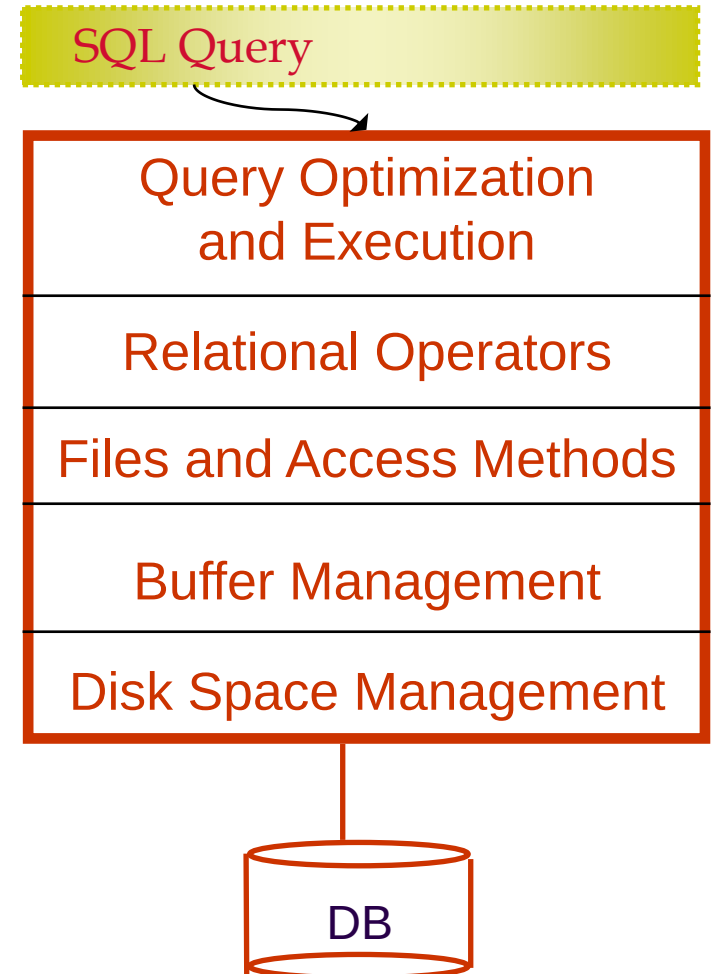
WEEK 2

QUERY PROCESSING PART I



Query Processing

- Some database operations are **EXPENSIVE**
- Can greatly improve performance by being “smart”
 - e.g., can speed up 1,000x over naïve approach
- Main weapons are:
 1. clever implementation techniques for **operators**
 2. exploiting “**equivalencies**” of relational operators
 3. using **statistics** and **cost models** to choose among these.



Cost-based Query Subsystem

Queries

```
Select *  
From Blah B  
Where B.blah = 'blah'
```

Query Parser

Query Optimizer

Plan
Generator

Plan Cost
Estimator

Query Plan Evaluator

Usually there is a
heuristics-based
rewriting step before
the cost-based steps.

Catalog Manager

Schema

Statistics

Basic Steps in Query Processing (Parsing)

- Parsing and translation
 - Translate the query into its internal form.
 - This is then translated into relational algebra.
 - Parser:
 - ▶ checks syntax,
 - ▶ verifies relations
- Optimization
 - Cheapest evaluation plan
- Evaluation
 - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.

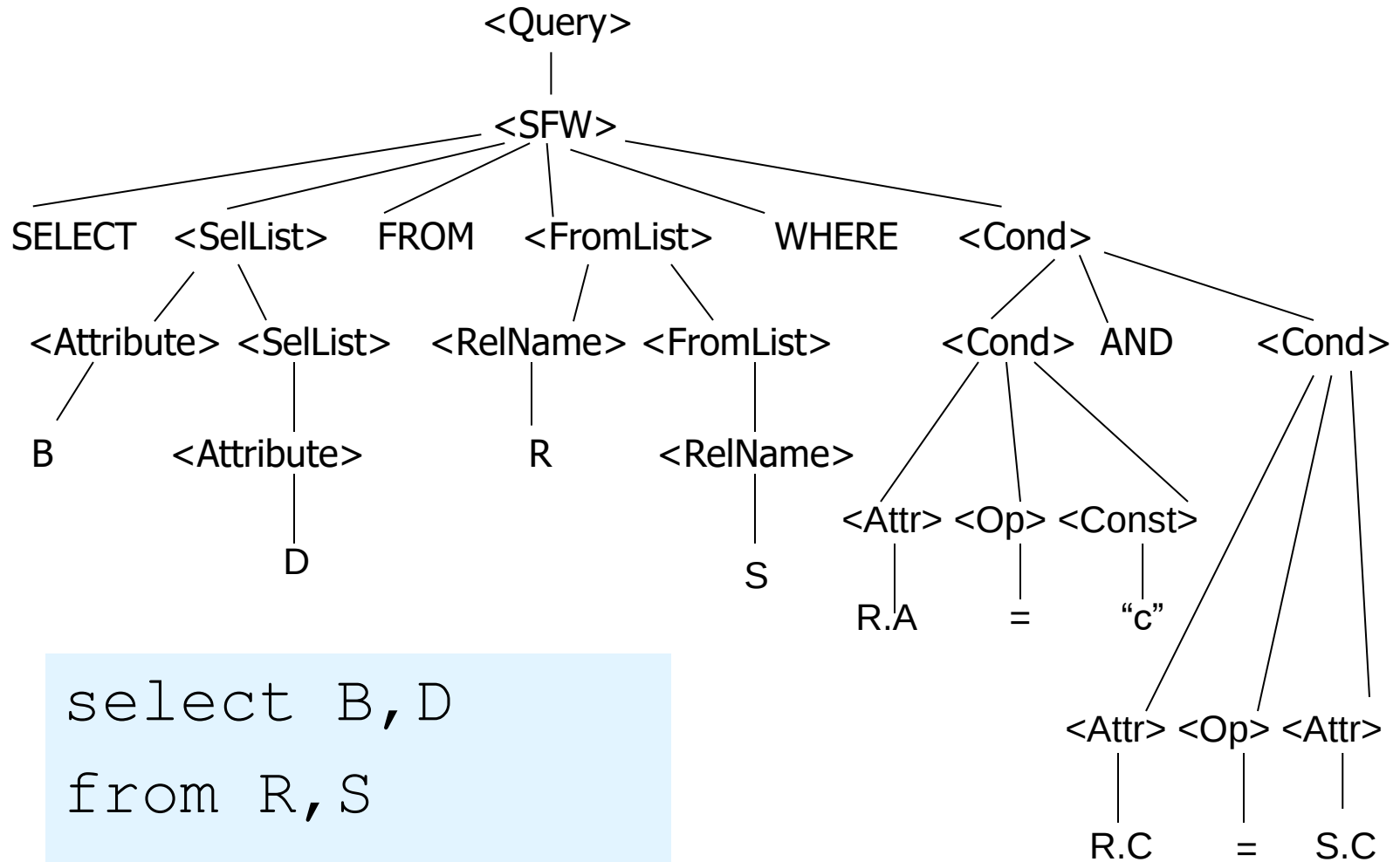


Example Query

```
select B,D  
from R,S  
where R.A = "c"  
       and R.C = S.C
```



Example: Parse Tree

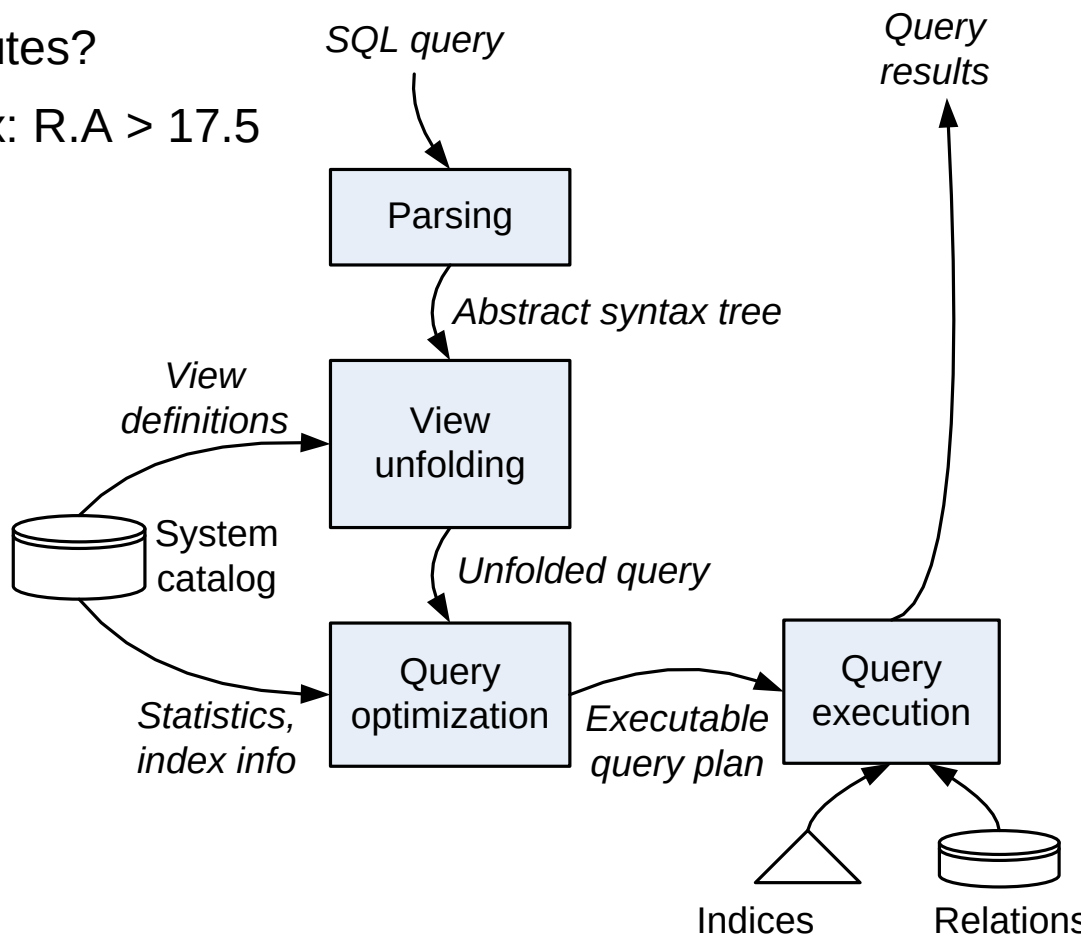


```
select B, D
from R, S
where R.A = "c"
and R.C = S.C
```



Along with Parsing ...

- Semantic checks
 - Do the projected attributes exist in the relations in the From clause?
 - Ambiguous attributes?
 - Type checking, ex: $R.A > 17.5$
- Expand **views**



Our Example Query in the DBMS Setting

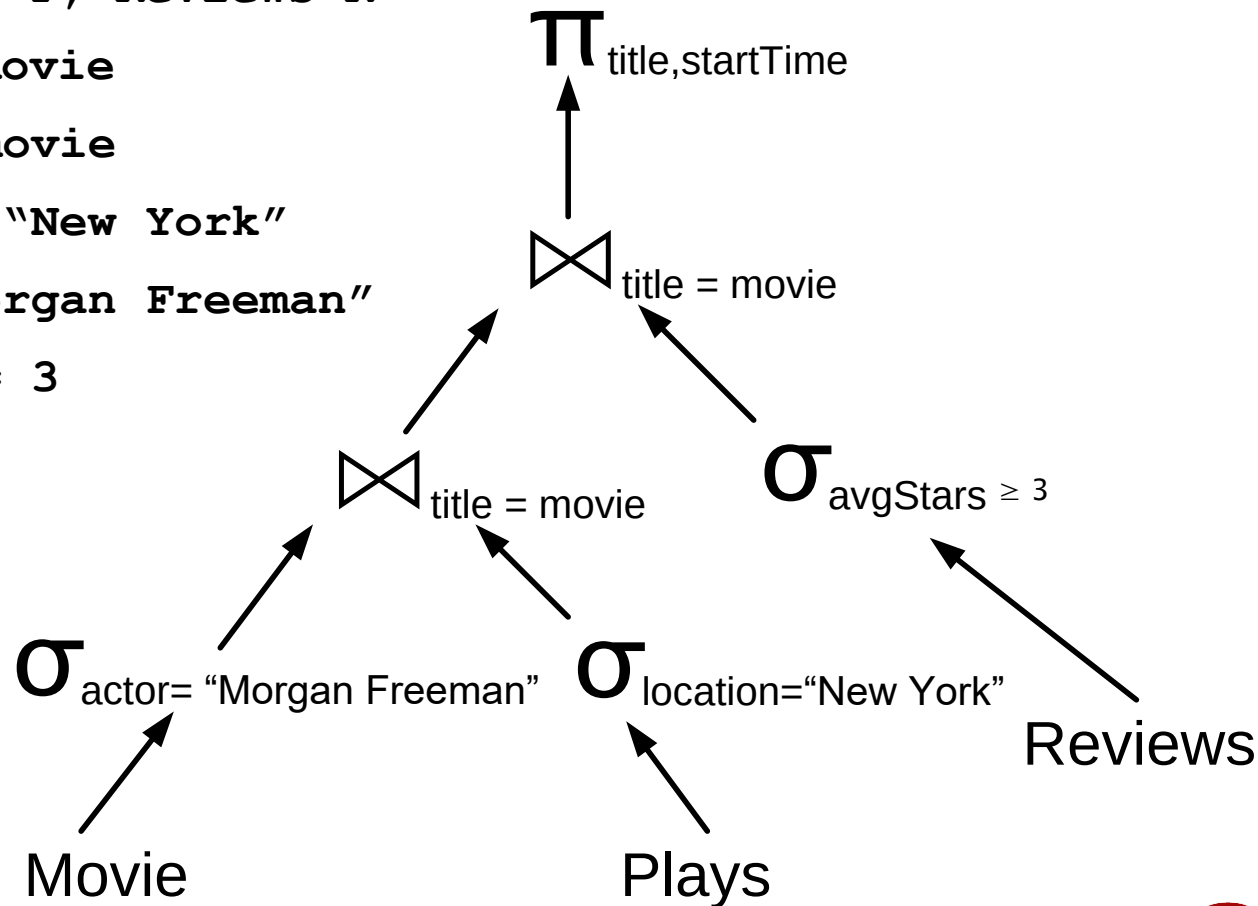
```
SELECT title, startTime
FROM Movie M, Plays P, Reviews R
WHERE M.title = P.movie
      AND M.title = R.movie
      AND P.location = "New York"
      AND M.actor = "Morgan Freeman"
      AND R.avgStars >= 3
```

- Query is parsed, generally broken into predicates, **unfolded as necessary**, then converted into a **logical query plan used by the optimizer...**



Example Logical Query Plan

```
SELECT title, startTime
FROM Movie M, Plays P, Reviews R
WHERE M.title = P.movie
      AND M.title = R.movie
      AND P.location = "New York"
      AND M.actor = "Morgan Freeman"
      AND R.avgStars >= 3
```

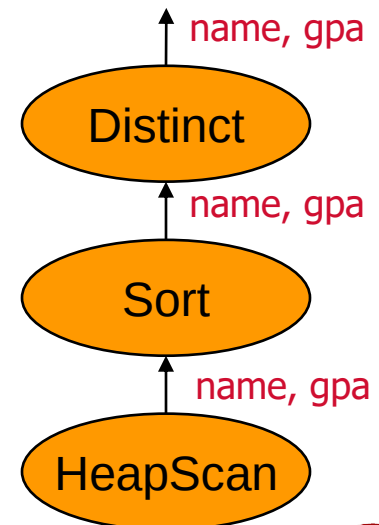
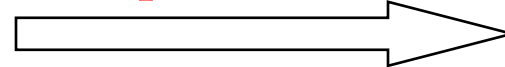


Query Processing Overview

- The *query optimizer* translates SQL to a special internal “language”
 - Query Plans
- The *query executor* is an *interpreter* for query plans
- Think of query plans as “box-and-arrow” *dataflow* diagrams
 - Each box implements a *relational operator*
 - Edges represent a flow of tuples (columns as specified)
 - For single-table queries, these diagrams are straight-line graphs

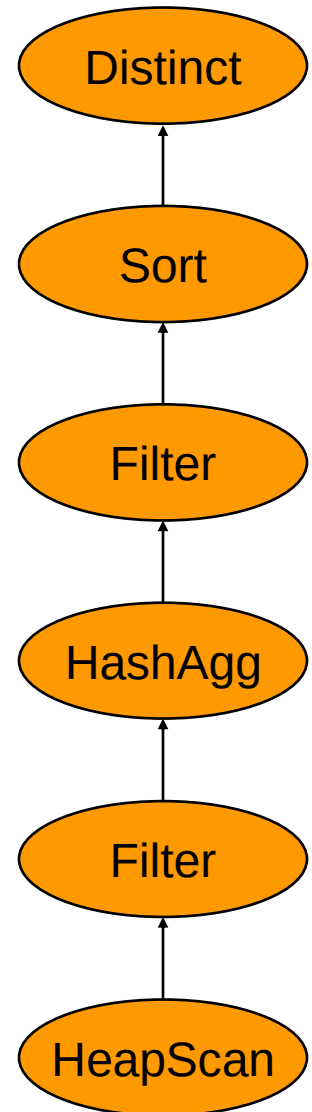
SELECT DISTINCT name, gpa
FROM Students

Optimizer



Query Optimization

- A deep subject, focuses on multi-table queries
 - We will only need a cookbook version for now.
- Build the dataflow bottom up:
 - Choose an Access Method (HeapScan or IndexScan)
 - ▶ Non-trivial, we'll learn about this later!
 - Next apply any WHERE clause filters
 - Next apply GROUP BY and aggregation
 - ▶ Can choose between sorting and hashing!
 - Next apply any HAVING clause filters
 - Next Sort to help with ORDER BY and DISTINCT
 - ▶ In absence of ORDER BY, can do DISTINCT via hashing!



Iterators

- The relational operators are all subclasses of the class `iterator`:

```
Class iterator {  
    void init();  
    tuple next();  
    void close();  
    iterator inputs[];  
  
    // additional state goes here  
}
```



iterator

- Note:
 - Edges in the graph are specified by inputs (max 2, usually 1)
 - Encapsulation: any iterator can be input to any other!
 - When subclassing, different iterators will keep different kinds of state information

Example: Scan Operator

```
Class Scan extends iterator {  
    void init();  
    tuple next();  
    void close();  
    iterator inputs[1];  
    bool_expr filter_expr;  
    proj_attr_list proj_list;  
}
```

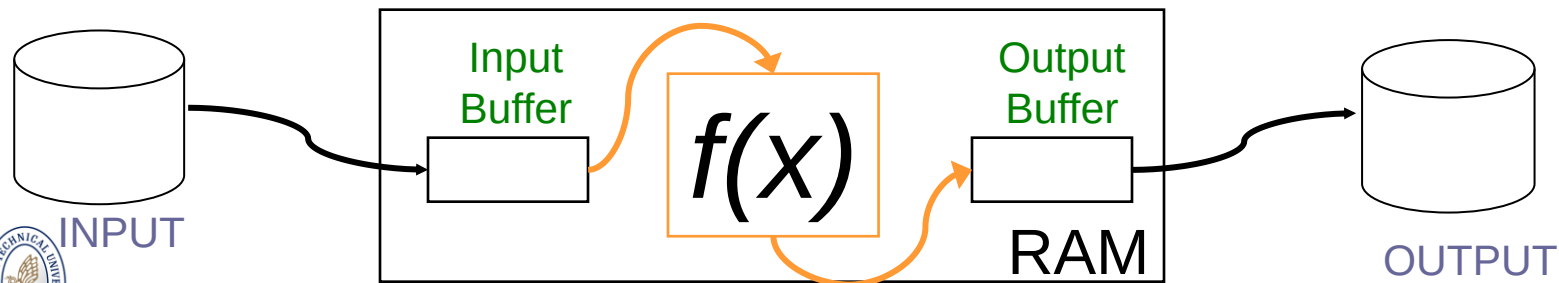
- `init()`:
 - set up internal state
 - call `init()` on child – often a file open
- `next()`:
 - call `next()` on child until qualifying tuple found or EOF
 - keep only those fields in “proj_list”
 - skip those that do not satisfy “filter_exp”
 - return tuple (or EOF -- “End of File” -- if no tuples remain)
- `close()`:
 - call `close()` on child
 - clean up internal state

Note that Scan also applies “selection” filters and “projections”
(without duplicate elimination)



Streaming through RAM

- Simple case: “Map” (assume many **records** per disk **page**)
 - Goal: Compute $f(x)$ for each record, write out the result
 - Challenge: minimize RAM, call read/write rarely
- Approach
 - Read a chunk from INPUT to an *Input Buffer*
 - Write $f(x)$ for each item to an *Output Buffer*
 - When Input Buffer is consumed, read another chunk
 - When Output Buffer fills, write it to OUTPUT
- Reads and Writes are **not coordinated** (i.e., not in lockstep)
 - e.g., if $f()$ is Compress(), you read many chunks per write.
 - e.g., if $f()$ is DeCompress(), you write many chunks per read.



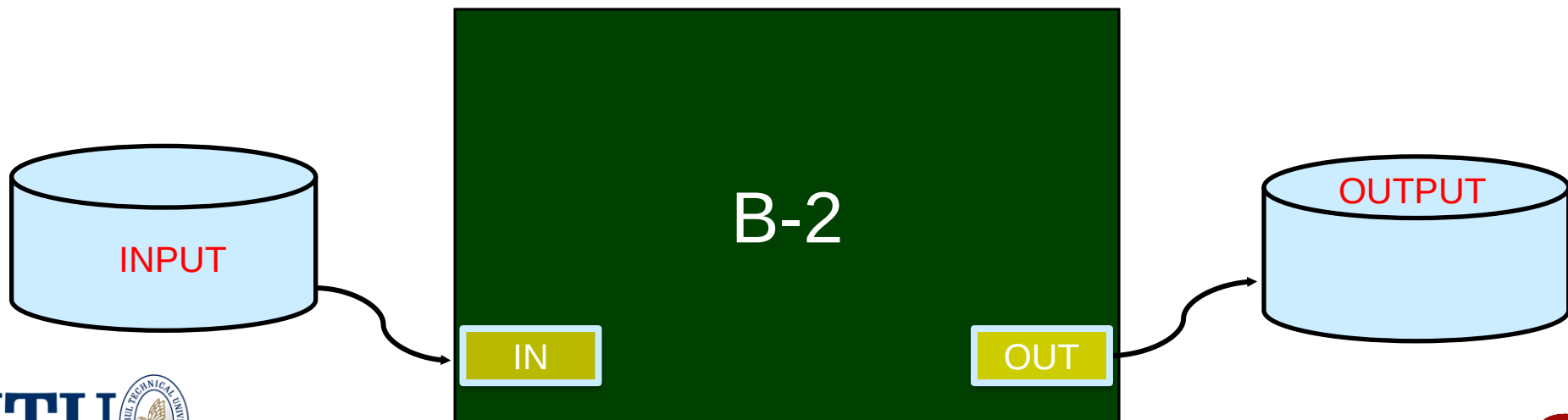
Rendezvous

- Streaming: one chunk at a time. Easy.
- But some algorithms need certain items to be co-resident in memory
 - not guaranteed to appear in the same input chunk
- *Time-space Rendezvous*
 - in the same place (RAM) at the same time
- There may be many combos of such items



Divide and Conquer

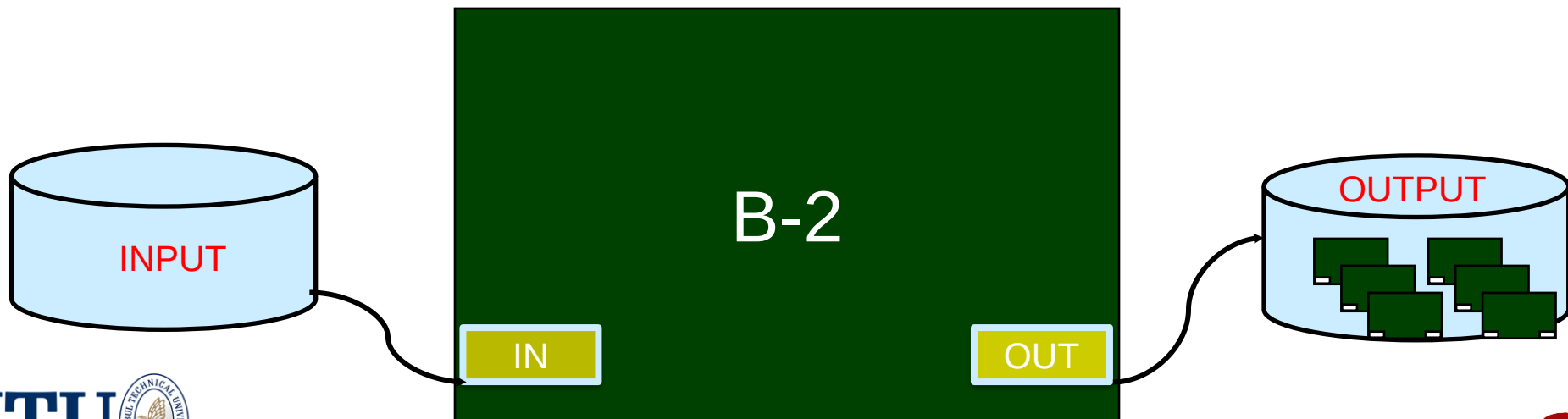
- *Out-of-core* algorithms orchestrate rendezvous.
- Typical RAM Allocation:
 - Assume B pages worth of RAM available
 - Use 1 page of RAM to read into (input buffer)
 - Use 1 page of RAM to write into (output buffer)
 - $B-2$ pages of RAM as workspace



Divide and Conquer

□ Phase 1

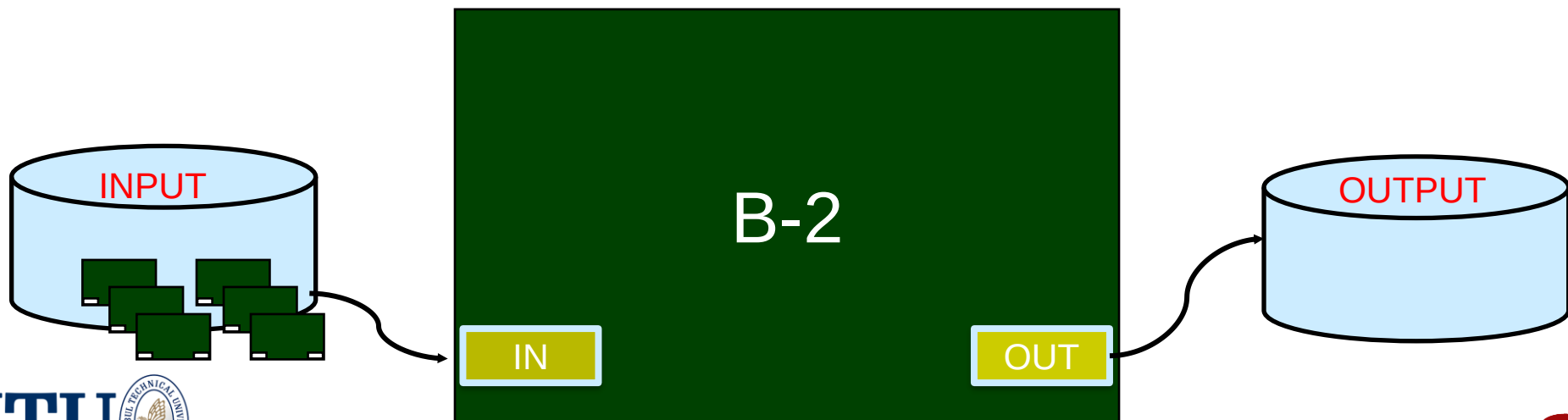
- “streamwise” *divide* into $N/(B-2)$ megachunks
- output (write) to disk one megachunk at a time



Divide and Conquer

□ Phase 2

- Now megachunks will be the input
- Process each *megachunk* individually.



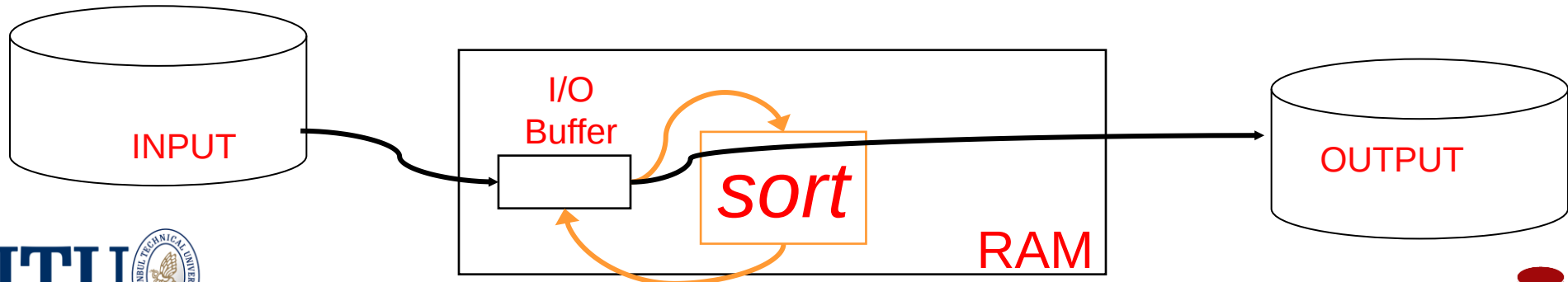
Example: Sorting Operation

- A classic problem in computer science!
- Data requested in sorted order
 - e.g., find students in increasing *gpa* order
- Sorting is useful for eliminating *duplicate copies* in a collection of records (?)
- *Sort-merge* join algorithm involves sorting.
- Sorting is first step in *bulk loading* B+ tree index.
- Problem: sort 100 TB of data with of 100 GB RAM.
 - Why not virtual memory?



Sorting: 2-Way

- Pass 0:
 - read a page, sort it, write it.
 - only one buffer page is used
 - a repeated “batch job”



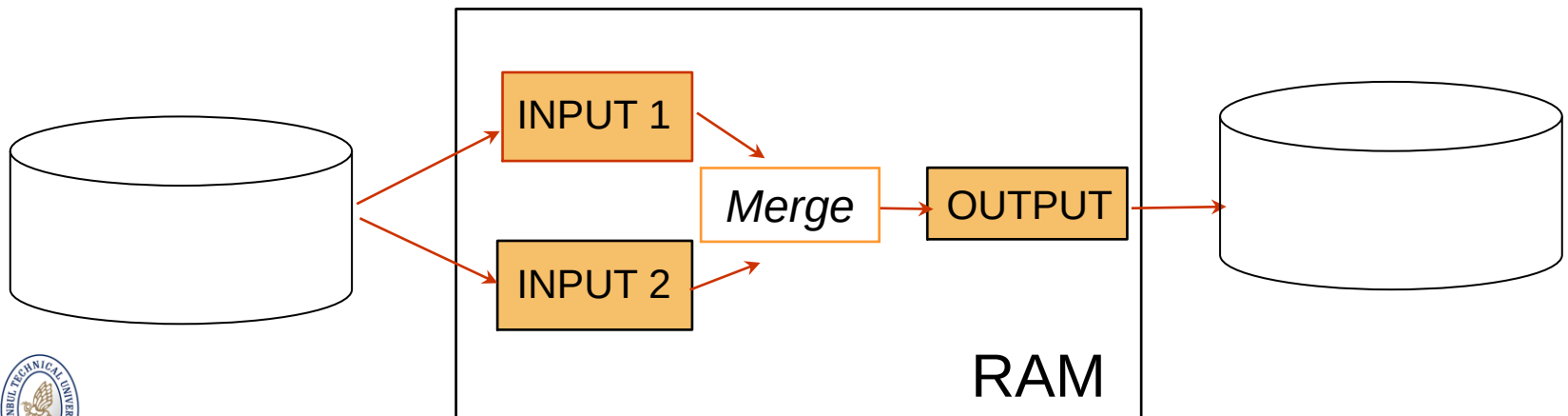
Internal Sort Algorithm

- ❑ Quicksort is a fast way to sort in memory.
- ❑ An alternative is “tournament sort” (a.k.a. “heapsort”)
 - ❑ **Top:** Read in B blocks and build a min-heap
 - ❑ **Output:** move smallest record to output buffer
 - ❑ Read in a new record r
 - ❑ insert r into “heap”
 - ❑ if r is not the smallest, then **GOTO Output**
 - ❑ else remove r from “heap”
 - ❑ output “heap” in order; **GOTO Top**

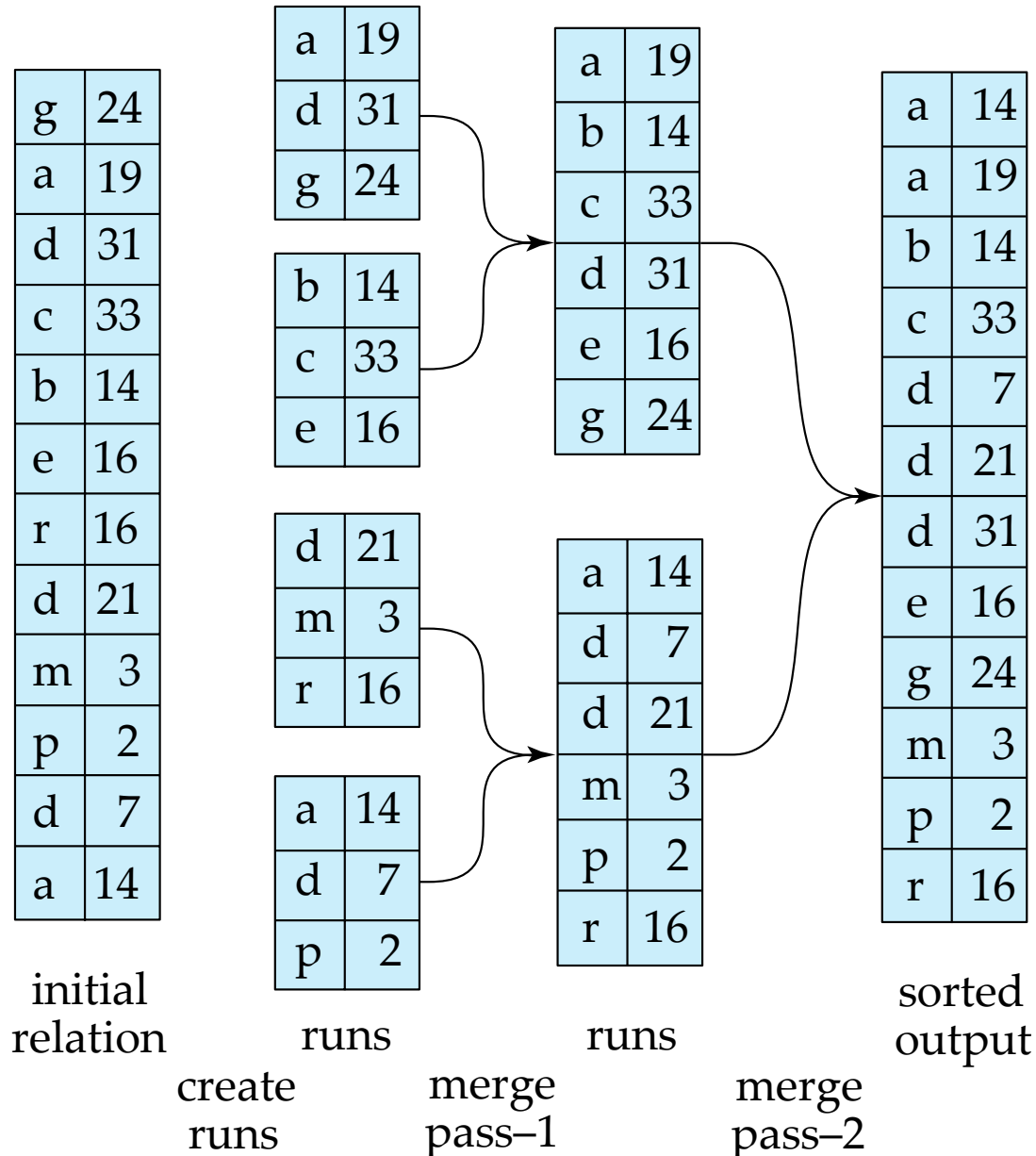


Sorting: 2-Way (cont.)

- Pass 1, 2, 3, ..., etc. (merge):
 - requires **3 buffer pages**
 - ▶ note: this has nothing to do with double buffering!
 - *merge pairs* of runs into runs twice as long
 - a streaming algorithm, as in the previous slide!



Example: External Sorting Using Sort-Merge



Two-Way External Merge Sort

Sort subfiles and Merge

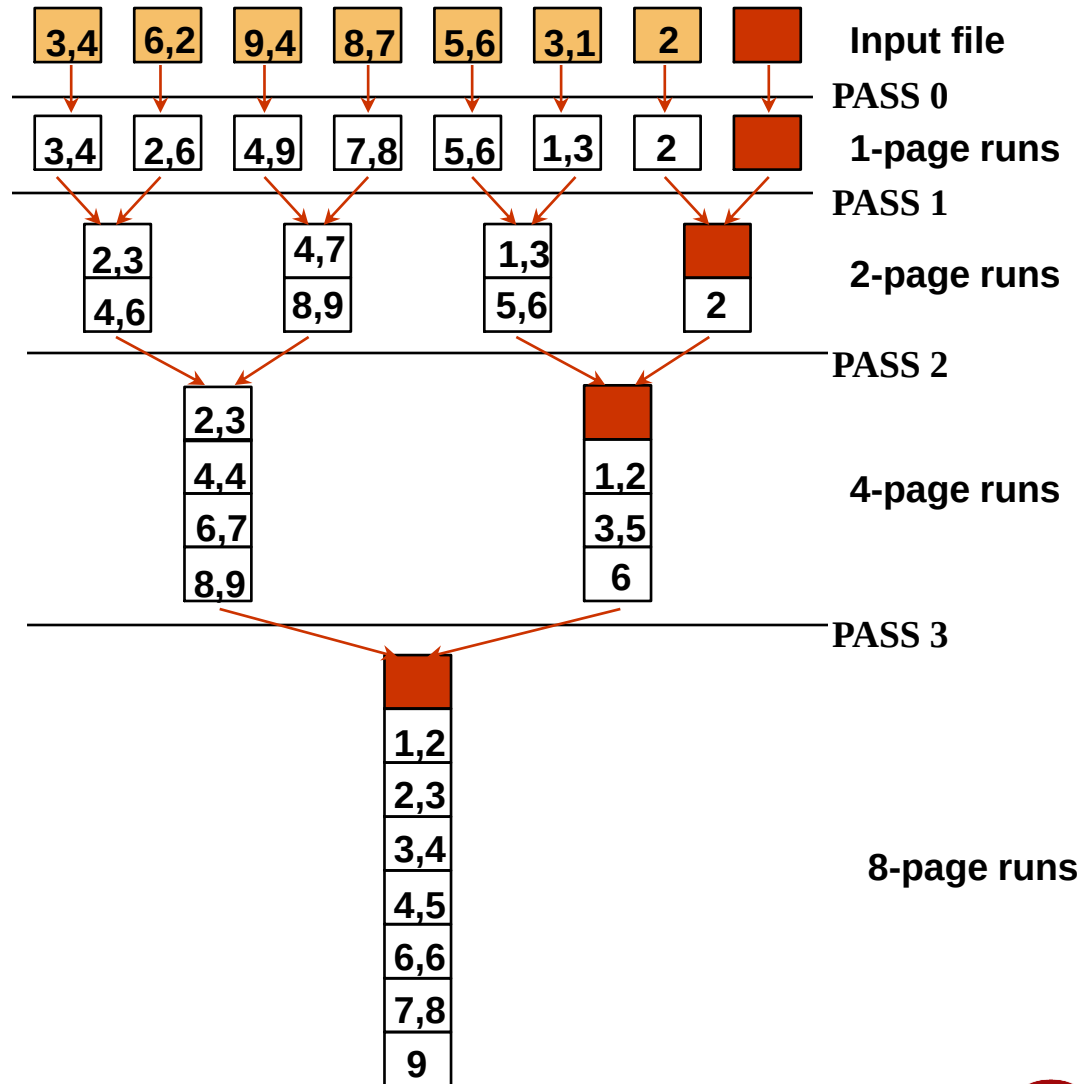
- How many passes?
- N pages in the file
=> the number of passes =

$$\lceil \log_2 N \rceil + 1$$

- Total I/O cost? (reads + writes)
- Each pass we read + write each page in file.

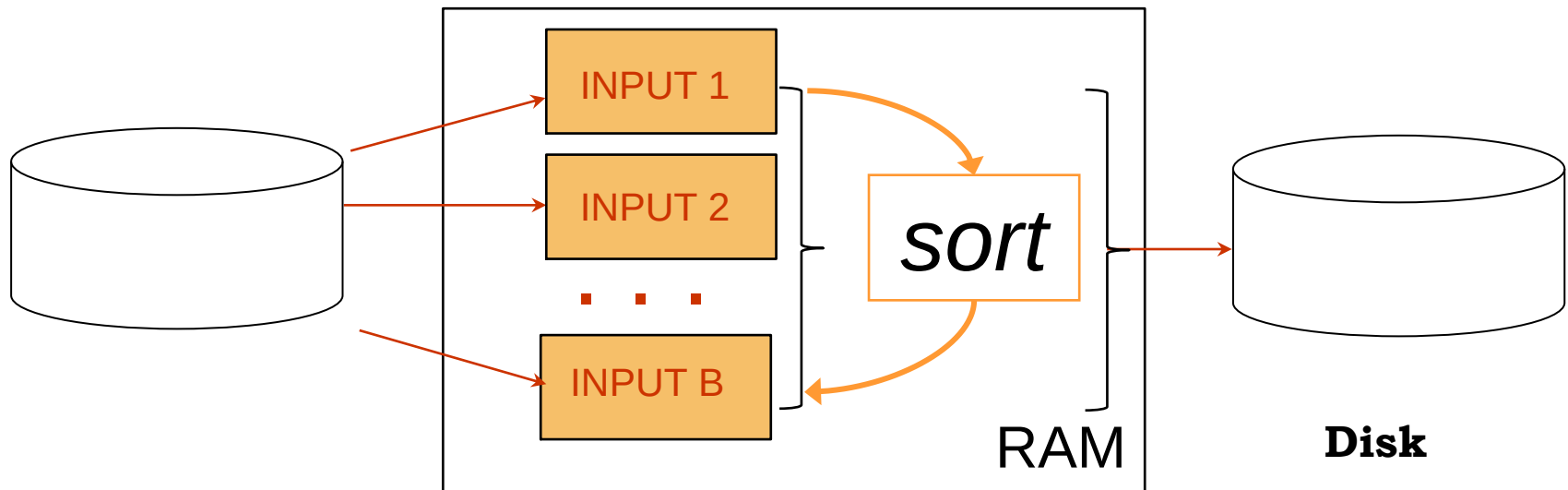
So total cost is:

$$2N(\lceil \log_2 N \rceil + 1)$$



General External Merge Sort

- More than 3 buffer pages. How can we utilize them?
- To sort a file with N pages using B buffer pages:
 - Pass 0: use B buffer pages. Produce $\lceil N / B \rceil$ sorted runs of B pages each.

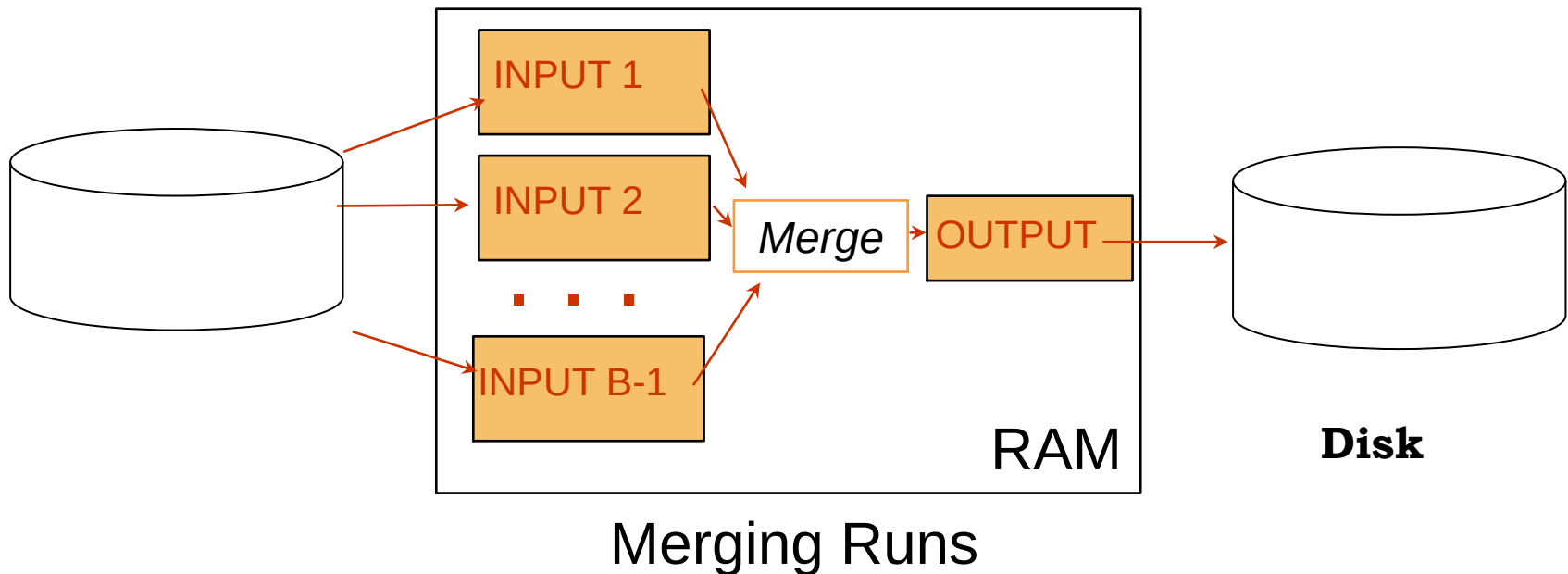


Pass 0 – Create Sorted Runs

General External Merge Sort

Pass 1, 2, ..., etc.: merge B-1 runs.

Creates runs of $(B-1) * \text{size of runs from previous pass (i.e., B)}$.



Cost of External Merge Sort

- Number of passes: $1 + \lceil \log_{B-1} \lceil N / B \rceil \rceil$
- Cost = $2N * (\# \text{ of passes})$
- E.g., with 5 buffer pages, to sort 108 page file:
 - Pass 0: $\lceil 108 / 5 \rceil = 22$ sorted runs of 5 pages each (last run is only 3 pages)
 - Pass 1: $\lceil 22 / 4 \rceil = 6$ sorted runs of 20 pages each (last run is only 8 pages)
 - Pass 2: 2 sorted runs, 80 pages and 28 pages
 - Pass 3: Sorted file of 108 pages

Formula check: $1 + \lceil \log_4 22 \rceil = 1 + 3 \rightarrow \underline{4 \text{ passes}} \quad \checkmark$



of Passes of External Sort

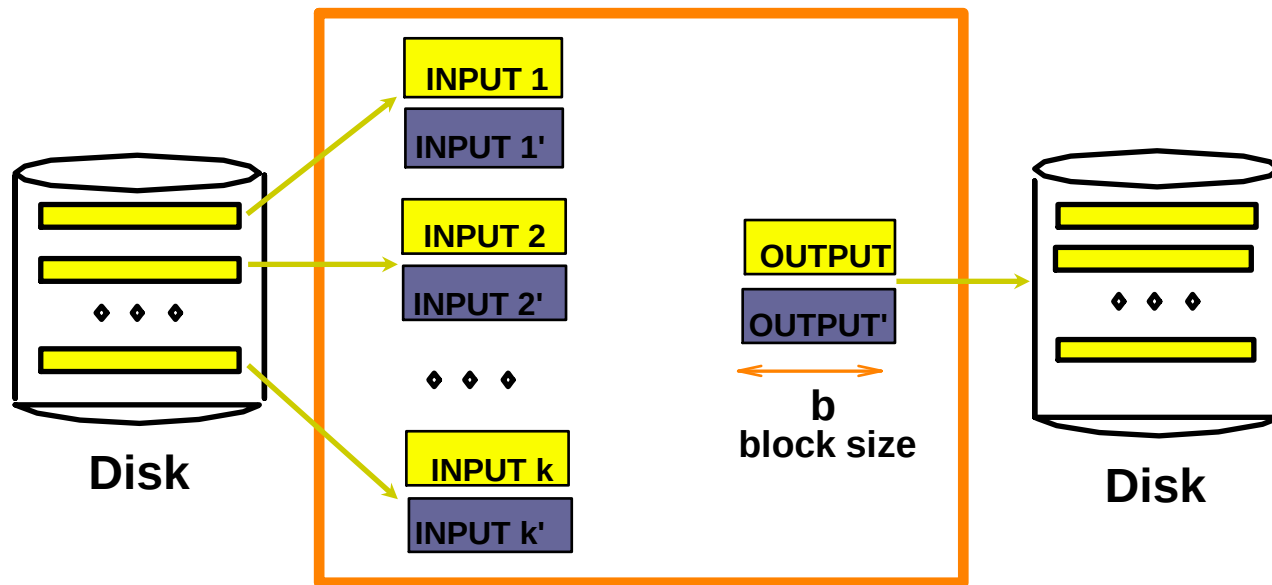
N	B=3	B=5	B=9	B=17	B=129	B=257
100	7	4	3	2	1	1
1,000	10	5	4	3	2	2
10,000	13	7	5	4	2	2
100,000	17	9	6	5	3	3
1,000,000	20	10	7	5	3	3
10,000,000	23	12	8	6	4	3
100,000,000	26	14	9	7	4	4
1,000,000,000	30	15	10	8	5	4

Then, I/O cost is $2N$ times number of passes.



Double Buffering

- To reduce wait time for I/O request to complete, can *prefetch* into 'shadow block'.
 - Potentially, more passes; in practice, most files still sorted in 2-3 passes.



B main memory buffers, k-way merge

Memory Requirement for External Sorting

- How big of a table can we sort in two passes?
 - Each “sorted run” after Phase 0 is of size B
 - Can merge up to B-1 sorted runs in Phase 1
- Answer: $B(B-1)$.
 - Sort N pages of data in about $\sqrt{N}$ space



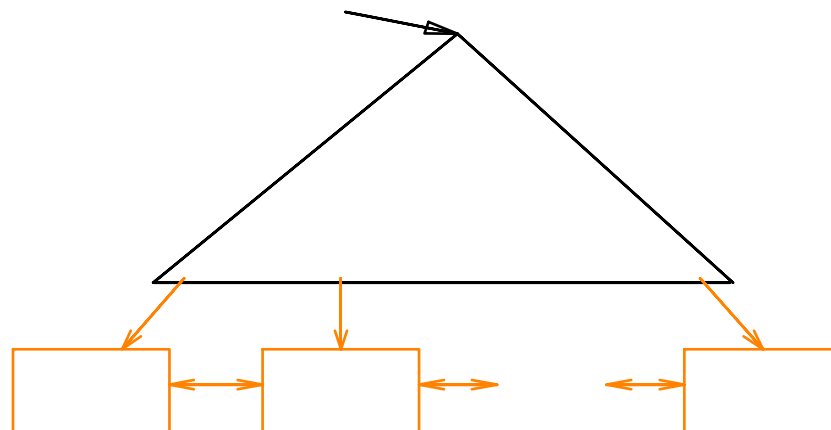
Using B+ Trees for Sorting

- ❑ Scenario: Table to be sorted has B+ tree index on sorting column(s).
- ❑ **Idea:** Can retrieve records in order by traversing leaf pages.
- ❑ ***Is this a good idea?***
- ❑ Cases to consider:
 - ❑ B+ tree is **clustered** ***Good idea!***
 - ❑ B+ tree is **not clustered** ***Could be a very bad idea!***



B+ Tree: Quick Overview

- Insert/delete at $\log_F N$ cost; keep tree *height-balanced*.
 - (F = fanout, N = # leaf pages)
- Minimum 50% occupancy (except for root).
- Each node contains $d \leq m \leq 2d$ entries.
 - d is called the *order* of the tree.
- Supports equality and range-searches efficiently.

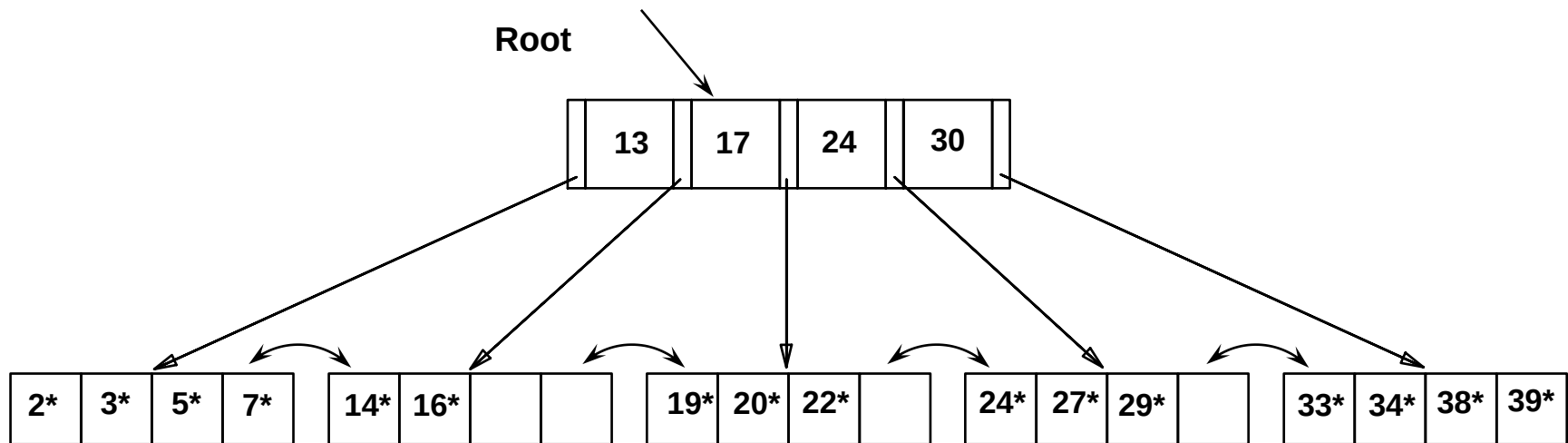


Index Entries
(Direct search)

Data Entries
("Sequence set")

Example B+ Tree

- Search begins at root, and key comparisons direct it to a leaf.
- Search for 5*, 15*, all data entries $\geq 24^*$...

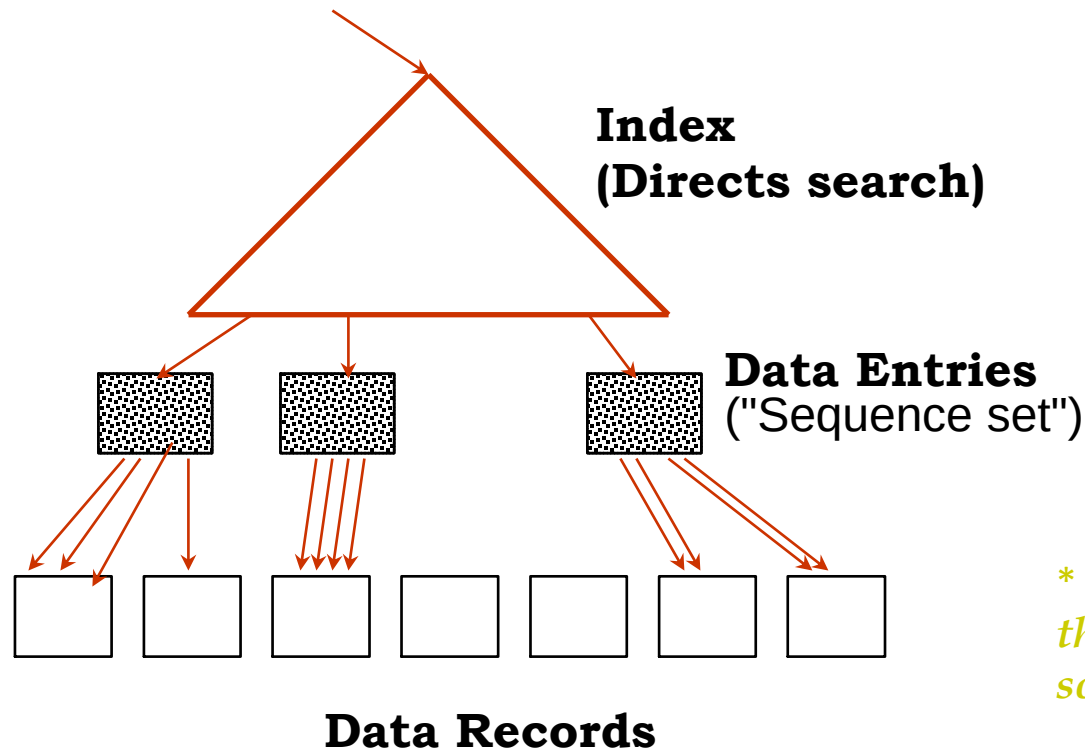


□ *Based on the search for 15*, we know it is not in the tree!*



Clustered B+ Tree Used for Sorting

- ❑ Cost: root to the left-most leaf, then retrieve all leaf pages (Alternative 1)



** Always better than external sorting!*



Unclustered B+ Tree Used for Sorting

- Alternative (2) for data entries; each data entry contains *rid* of a data record.
- In general, **one I/O per data record!**

